

AI ASSISTED CODING

ASSIGNMENT-10.1

NAME: Varshitha. G

HTNO: 2303A51103

Task Description #1 – Syntax and Logic Errors

Task: Use AI to identify and fix syntax and logic errors in a faulty Python script.

Sample Input Code:

```
# Calculate average score of a student

def calc_average(marks):

    total = 0

    for m in marks:

        total += m

    average = total / len(marks)

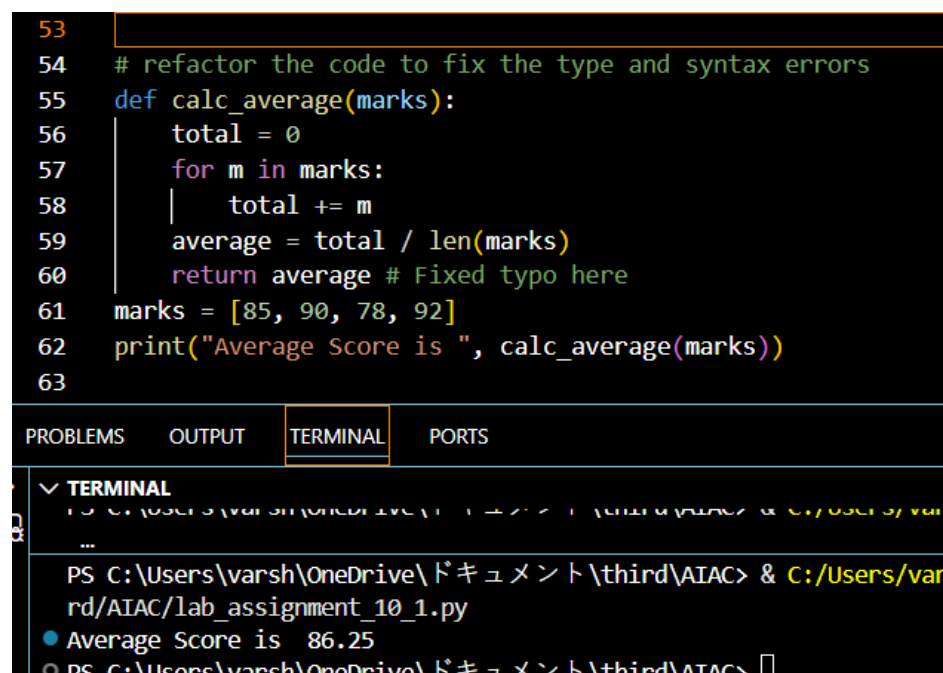
    return avrage # Typo here

marks = [85, 90, 78, 92]

print("Average Score is ", calc_average(marks))
```

Expected Output:

- Corrected and runnable Python code with explanations of the fixes.



```
53
54 # refactor the code to fix the type and syntax errors
55 def calc_average(marks):
56     total = 0
57     for m in marks:
58         total += m
59     average = total / len(marks)
60     return average # Fixed typo here
61 marks = [85, 90, 78, 92]
62 print("Average Score is ", calc_average(marks))
63
```

PROBLEMS OUTPUT **TERMINAL** PORTS

▼ **TERMINAL**

```
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/OneDrive/ドキュメント\third\AIAC/lab_assignment_10_1.py
Average Score is 86.25
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

Typo corrected – Changed return avrage to return average.

Missing bracket fixed – Added the closing) in the print() statement.

Task Description #2 – PEP 8 Compliance

Task: Use AI to refactor Python code to follow PEP 8 style guidelines.

Sample Input Code:

```
def area_of_rect(L,B) : return L*B  
  
print(area_of_rect(10,20))
```

Expected Output:

- Well-formatted PEP 8-compliant Python code.

```
3 # refactor the above code to enhance readability and maintainability well commented and add type hints  
4 def area_of_rectangle(length: float, breadth: float) -> float:  
5     """  
6     This function calculates the area of a rectangle given its length and breadth.  
7  
8     Parameters:  
9     length (float): The length of the rectangle  
0     breadth (float): The breadth of the rectangle  
1  
2     Returns:  
3     float: The area of the rectangle  
4     """  
5     return length * breadth  
6 # Example usage:  
7 length = 10.0  
8 breadth = 20.0  
9 area = area_of_rectangle(length, breadth)  
0 print(f"The area of the rectangle with length {length} and breadth {breadth} is: {area}")  
1
```

PROBLEMS OUTPUT **TERMINAL** PORTS

TERMINAL

KeyboardInterrupt
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/python
rd/AIAC/lab_assignment_10_1.py
The area of the rectangle with length 10.0 and breadth 20.0 is: 200.0
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> █

Task Description #3 – Readability Enhancement

Task: Use AI to make code more readable without changing its logic.

Sample Input Code:

```
def c(x,y):  
  
    return x*y/100  
  
a=200  
  
b=15
```

```
print(c(a,b))
```

Expected Output:

- Python code with descriptive variable names, inline comments, and clear formatting.

```
8 def calculate_percentage(part: float, whole: float) -> float:
9     """
10     This function calculates the percentage of a part relative to a whole.
11
12     Parameters:
13     part (float): The portion or part of the whole
14     whole (float): The total or whole amount
15
16     Returns:
17     float: The percentage value of the part relative to the whole
18     """
19     return (part * 100) / whole
20
21 # Example usage:
22 numerator = 200
23 denominator = 15
24 percentage = calculate_percentage(numerator, denominator)
25 print(f"The percentage of {numerator} relative to {denominator} is: {percentage}%")
```

PROBLEMS	OUTPUT	TERMINAL	PORTS
✓ TERMINAL			
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python311/Python.exe C:\Users\varsh\OneDrive\ドキュメント\third\AIAC\percentage.py			
The percentage of 200 relative to 15 is: 1333.3333333333333%			
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> █			

Task Description #4 – Refactoring for Maintainability

Task: Use AI to break repetitive or long code into reusable functions.

Sample Input Code:

```
students = ["Alice", "Bob", "Charlie"]
```

```
print("Welcome", students[0])
```

```
print("Welcome", students[1])
```

```
print("Welcome", students[2])
```

Expected Output:

- Modular code with reusable functions.

```
78 # Break repetitive or long code into reusable functions. Modular code with reusable functions
79 def welcome_students(student_list):
80     """
81     This function takes a list of student names and prints a welcome message for each student.
82
83     Parameters:
84     student_list (list): A list of student names
85
86     Returns:
87     None
88     """
89     for student in student_list:
90         print(f"Welcome {student}")
91 # Example usage:
92 students = ["Alice", "Bob", "Charlie"]
93 welcome_students(students)
94
```

PROBLEMS	OUTPUT	TERMINAL	PORTS
----------	--------	----------	-------

✓ **TERMINAL**

```
○ Welcome Alice
Welcome Bob
Welcome Charlie
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

Task Description #5 – Performance Optimization

Task: Use AI to make the code run faster.

Sample Input Code:

```
# Find squares of numbers

nums = [i for i in range(1,1000000)]

squares = []

for n in nums:

    squares.append(n**2)

print(len(squares))
```

Expected Output:

- Optimized code using list comprehensions or vectorized operations.

```
22 |
23 import time
24 time1=time.time()
25 nums = [i for i in range(1,1000000)]
26 squares = []
27 for n in nums:
28     squares.append(n**2)
29 print(len(squares))
30 time2=time.time()
31
32 # refactor the above code with less time complexity
33
34 time3=time.time()
35 squares = [n**2 for n in range(1, 1000000)]
36 print(len(squares))
37
38 time4=time.time()
39 print(f"Time taken for original code: {time2 - time1} seconds")
40 print(f"Time taken for refactored code: {time4 - time3} seconds")
41
--
PROBLEMS OUTPUT TERMINAL PORTS
▼ TERMINAL
○ PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python39-64/Python.exe C:\Users\varsh\OneDrive\ドキュメント\third\AIAC\main.py
999999
999999
Time taken for original code: 0.10435295104980469 seconds
Time taken for refactored code: 0.06417250633239746 seconds
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> 
```

Task Description #6 – Complexity Reduction

Task: Use AI to simplify overly complex logic.

Sample Input Code:

```
def grade(score):
```

```
    if score >= 90:
```

```
        return "A"
```

```
    else:
```

```
        if score >= 80:
```

```
            return "B"
```

```
    else:
```

```
        if score >= 70:
```

```
return "C"
```

```
else:
```

```
if score >= 60:
```

```
return "D"
```

```
else:
```

```
return "F"
```

Expected Output:

- Cleaner logic using elif or dictionary mapping.

```
5 # simplify above complex logic make the code with cleaner logic
7 def grade(score):
8     if score >= 90:
9         return "A"
10    elif score >= 80:
11        return "B"
12    elif score >= 70:
13        return "C"
14    elif score >= 60:
15        return "D"
16    else:
17        return "F"
18 print(grade(95)) # Output: A
19 print(grade(85)) # Output: B
20
```

PROBLEMS	OUTPUT	TERMINAL	PORTS
TERMINAL			
A			
B			
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>			